

EXP.N0:2 _IMPLEMENTATION OF PLAYFAIR CIPHER

AIM:

To write a C program to implement the Playfair Substitution technique.

ALGORITHM:

STEP-1: Read the plain text from the user.

STEP-2: Read the keyword from the user.

STEP-3: Arrange the keyword without duplicates in a 5*5 matrix in the row order and fill the remaining cells with missed out letters in alphabetical order. Note that „i“ and „j“ takes the same cell.

STEP-4: Group the plain text in pairs and match the corresponding corner letters by forming a rectangular grid.

STEP-5: Display the obtained cipher text.

PROGRAM:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
#define SIZE 5
```

```
char keyMatrix[SIZE][SIZE];
```

```
void generateKeyMatrix(char key[])
```

```
{    int used[26] = {0};    int i,
```

```
j = 0, k = 0;
```

```
    used['J' - 'A'] = 1;
```

```
    for (i = 0; key[i] != '\0'; i++)
```

```
    {        char ch = toupper(key[i]);
```

```
    if (ch == 'J')        ch = 'I';
```

```
        if (!used[ch - 'A'])
```

```
        {
```

```
            keyMatrix[j][k++] = ch;        used[ch -
```

```
'A'] = 1;
```

```
        if (k == SIZE)
```

```
        {            k = 0;
```

```
j++;        }
```

```
    }
```

```
}
```

```
for (i = 0; i < 26; i++)
```

```
{    if (!used[i])
```

```
{
```

```
keyMatrix[j][k++] = i + 'A';
```

```
if (k == SIZE)
```

```
{           k = 0;
```

```
j++;
```

```
}
```

```
}
```

```
}
```

```
}
```

```
void findPosition(char ch, int *row, int *col)
```

```
{
```

```
int i, j;
```

```
if (ch == 'J')    ch = 'I';
```

```
for (i = 0; i < SIZE; i++)
```

```
{    for (j = 0; j < SIZE; j++) {
```

```
if (keyMatrix[i][j] == ch)
```

```
{
```

```
    *row = i;    *col
```

```
= j;    return;
```

```
}
```

```
    }  
    }  
}
```

```
void encrypt(char a, char b)
```

```
{    int r1, c1, r2, c2;
```

```
    findPosition(a, &r1, &c1);    findPosition(b, &r2,  
&c2);
```

```
    if (r1 == r2)
```

```
    {
```

```
        printf("%c%c", keyMatrix[r1][(c1 + 1) % 5], keyMatrix[r2][(c2 + 1) % 5]);
```

```
    }    else if (c1 == c2)
```

```
    {
```

```
        printf("%c%c", keyMatrix[(r1 + 1) % 5][c1], keyMatrix[(r2 + 1) % 5][c2]);
```

```
    }    else
```

```
    {
```

```
        printf("%c%c", keyMatrix[r1][c2], keyMatrix[r2][c1]);
```

```
    }
```

```
}
```

```

int main() {    char key[100],
text[100];

    int i;


    printf("Enter Key: ");    scanf("%s", key);


    printf("Enter Plain Text: ");    scanf("%s", text);


    generateKeyMatrix(key);


    printf("\nKey Matrix:\n");    for (i = 0; i
< 5; i++)
    {
        int j;        for (j = 0; j < 5; j++)
printf("%c ", keyMatrix[i][j]);        printf("\n");
    }


    printf("\nCipher Text: ");


    for (i = 0; text[i] != '\0'; i += 2)

    {        char a = toupper(text[i]);        char b = (text[i + 1] != '\0') ?
toupper(text[i + 1]) : 'X';

```

```
        if (a == 'J') a = 'I';        if (b
== 'J') b = 'I';
```

```
        if (a == b)
encrypt(a, 'X');        else
encrypt(a, b);
    }
```

```
    return 0;
}
```

OUTPUT:

OutputClear

```
Enter Key: ciper
Enter Plain Text: Avinash

Key Matrix:
C I P E R
A B D F G
H K L M N
O Q S T U
V W X Y Z

Cipher Text: HCRKDOLV

=== Code Execution Successful ===
```

RESULT:

Thus the Playfair cipher substitution technique had been implemented successfully.